

E-COMMERCE · ARCHITECTURE & IMPLEMENTATION

Recommendation Systems: Architecture & Implementation

Retrieval, ranking, serving, and evaluation at scale

A technical deep-dive for the engineering team

Presented by Your Name · June 2026



Speaker notes

Audience is engineers and ML practitioners, so we assume fluency and spend our time on system design, modeling choices, infra, and evaluation. The deck mirrors a real production stack: problem framing → reference architecture → retrieval → ranking → data/features → cold start → evaluation → serving/latency → production case studies → challenges → stack/build decisions.

MODIFIABLE FOR YOUR PLATFORM

- Swap in presenter name and date.
- Tailor the title to the specific system or service you're proposing to build.

PROBLEM FRAMING

Top-K over $f(\text{user}, \text{item}, \text{context})$ under tight constraints

Goal: rank the catalog by predicted relevance and return the top-K, where $\text{score} = f(\text{user}, \text{item}, \text{context})$.

**Latency**

Score & rank within a page-load budget (tens of ms).

**Scale**

Catalogs of 10^6 – 10^8 items; millions of QPS at peak.

**Freshness**

New items, new behavior, and context must propagate fast.

**Objective**

Optimize a business metric, not just offline accuracy.

Speaker notes

Frame recommendation as constrained top-K retrieval over a scoring function. The four constraints drive every later decision: latency forces multi-stage retrieval (you can't score 10^8 items with a deep net per request); scale forces approximate methods; freshness forces a real-time path alongside batch; and the objective being a business metric is why offline accuracy alone is insufficient. Keep returning to these constraints when justifying architecture choices.

MODIFIABLE FOR YOUR PLATFORM

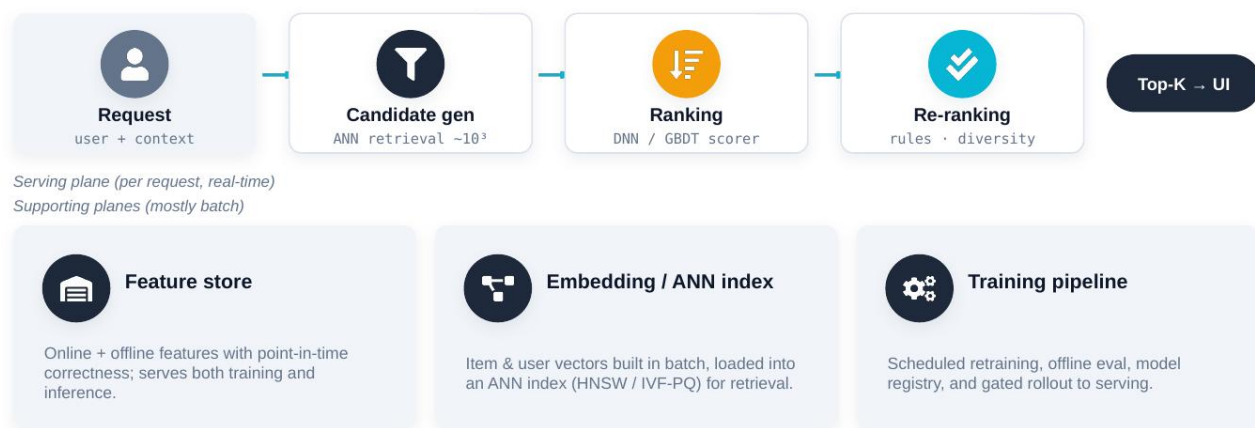
- Replace the catalog size (10^6 – 10^8) and QPS with your platform's actual numbers.
- State your real latency budget (e.g., p99 < 80 ms end-to-end).

ABBREVIATIONS

- QPS = Queries Per Second
- K = the number of items returned (top-K)
- ms = milliseconds

REFERENCE ARCHITECTURE

A multi-stage funnel with separate serving & training planes



Speaker notes

This is the canonical production shape — walk the serving funnel top, supporting planes bottom. Per request: resolve user+context, retrieve $\sim 10^3$ candidates via ANN, score them with a heavier ranker, then re-rank with business rules, return top-K. Underneath: a feature store with point-in-time correctness (critical to avoid train/serve skew), an embedding/ANN index built in batch, and a training pipeline with offline eval and gated rollout. The key design principle is decoupling expensive computation (batch) from cheap serving (online lookup).

MODIFIABLE FOR YOUR PLATFORM

- Annotate each box with the actual technology you use or propose (e.g., Feast for the feature store, FAISS/ScaNN for ANN).

ABBREVIATIONS

- ANN = Approximate Nearest Neighbor (fast similarity search)
- HNSW / IVF-PQ = ANN index structures
- DNN = Deep Neural Network
- GBDT = Gradient-Boosted Decision Trees

STAGE 1 — CANDIDATE GENERATION

High-recall retrieval, cheap per item

Reduce 10^6 – 10^8 items to $\sim 10^3$ candidates in milliseconds. Optimize recall; precision is the ranker's job.



Two-tower retrieval

Separate user & item encoders; dot-product in shared space; ANN at serve time.



Graph embeddings

Item graph from behavior sequences (e.g., Alibaba EGES) with side info for cold start.



Co-occurrence

Market-basket / item-to-item from co-purchase & co-view logs.



Sources & blending

Multiple recall sources (recent, popular, personalized) merged and de-duped.

ANN backends: HNSW, IVF-PQ, ScaNN — tune recall vs. latency vs. memory.

Speaker notes

Candidate generation is a recall problem: you must not drop good items, but you can tolerate junk because the ranker filters it. Two-tower is the modern default — precompute item embeddings, embed the user/context at request time, ANN lookup. Graph embeddings (Alibaba's EGES is the canonical reference) handle huge sparse catalogs and incorporate side information for cold start. Co-occurrence is the cheap, strong baseline. In practice you blend several recall sources. Backend choice (HNSW/IVF-PQ/ScaNN) is a recall/latency/memory tradeoff.

ABBREVIATIONS

- EGES = Enhanced Graph Embedding with Side information (Alibaba/Taobao)
- ANN = Approximate Nearest Neighbor
- HNSW, IVF-PQ, ScaNN = ANN index implementations

STAGE 2 — RANKING MODELS

From linear models to sequence-aware deep nets



Inputs: dense features + sparse-ID embeddings. Train on implicit feedback (clicks/purchases) with negative sampling.

Speaker notes

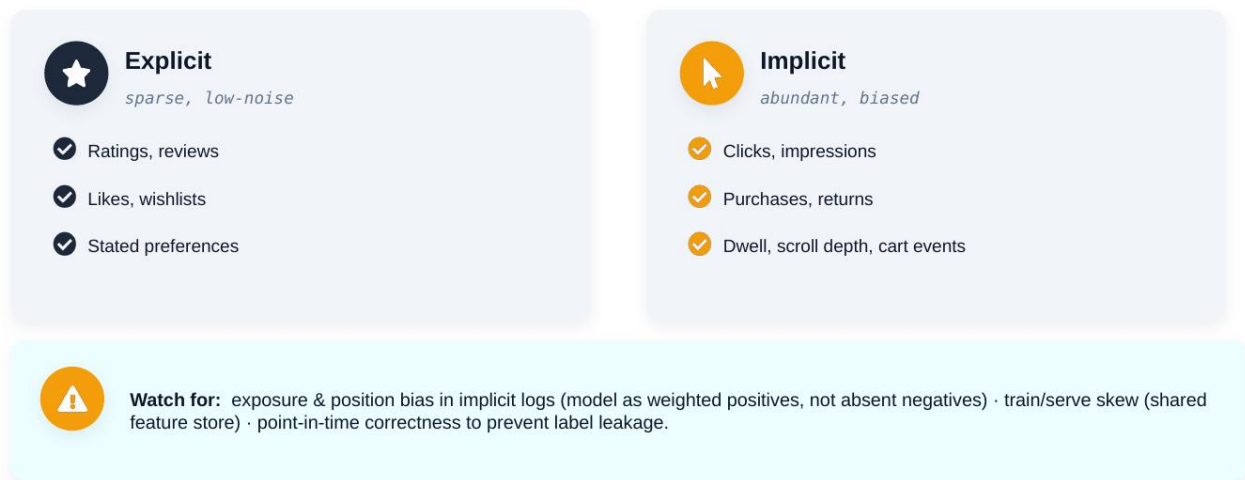
Show the trajectory without dismissing the simple options — GBDTs remain competitive and cheap, and are still widely deployed for ranking. Wide & Deep introduced combining memorization with generalization; DIN added attention over user history; DLRM-style models scale sparse embedding tables; SASRec and transformer rankers model the interaction sequence for session intent. The common thread: dense + sparse-embedding inputs, trained on implicit feedback with negative sampling. Pick the simplest model that hits your metric and latency budget.

ABBREVIATIONS

- LR = Logistic Regression
- GBDT = Gradient-Boosted Decision Trees
- DIN = Deep Interest Network
- DLRM = Deep Learning Recommendation Model
- SASRec = Self-Attentive Sequential Recommendation

DATA & FEATURE PIPELINE

Garbage in, garbage ranked



Speaker notes

The data work is where most of the wins (and bugs) live. Explicit signals are clean but sparse; implicit dominates volume but is biased — users can only click what was shown (exposure/position bias), so treat implicit feedback as weighted positives rather than clean labels. Two infra musts: a shared feature store to eliminate train/serve skew, and point-in-time correctness so you never train on information that wouldn't have been available at prediction time (label leakage).

MODIFIABLE FOR YOUR PLATFORM

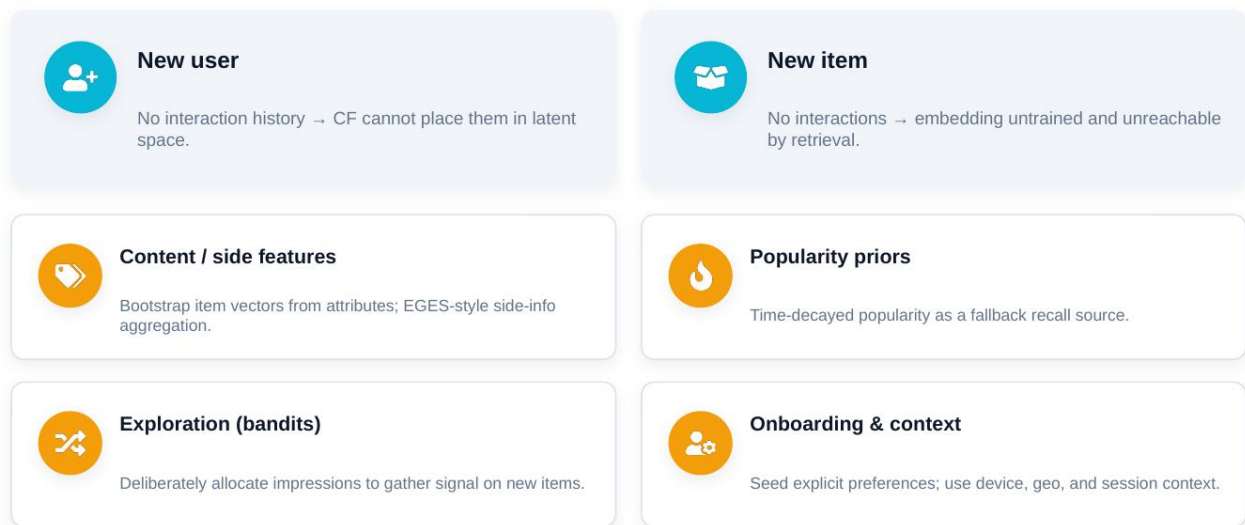
- List the actual event streams and feature store you use.
- Note your logging of impressions/positions, which is required for debiasing.

ABBREVIATIONS

- Train/serve skew = mismatch between features at training vs. inference time

COLD START, TECHNICALLY

Bootstrap entities with no interaction vector



Speaker notes

Frame cold start as a missing-embedding problem. For new items, side-information aggregation — exactly what EGES does by folding category/brand/price into the graph embedding — gives a usable vector before any interactions exist. Popularity priors are the safe fallback. Exploration (multi-armed/contextual bandits) is the principled way to buy signal on new items rather than waiting passively. For new users, onboarding and context features seed the vector.

ABBREVIATIONS

- CF = Collaborative Filtering
- EGES = Enhanced Graph Embedding with Side information
- Bandit = exploration algorithm balancing trying new options vs. exploiting known-good ones

EVALUATION METHODOLOGY

Offline gates; online decides

Offline (proxy)

- Precision@k, Recall@k — retrieval quality
- NDCG, MAP — ranking quality (top-weighted)
- AUC — pairwise ranking / CTR calibration
- Fast & cheap, but only a proxy for impact

Online (ground truth)

- A/B tests with statistical significance
- Interleaving for sensitive ranking comparisons
- Primary: CTR, CVR, revenue / AOV
- Guardrails: latency, diversity, returns

Mind the offline/online gap: better NDCG ≠ better revenue. Validate every launch online.

Speaker notes

Be explicit about the two-tier evaluation. Offline metrics (precision@k, recall@k, NDCG, MAP, AUC) are cheap and used to gate candidate models, but they're proxies. Online — A/B testing with proper significance, and interleaving for sensitive ranking comparisons — is the ground truth, judged on business metrics with guardrails (latency, diversity, return rate). The headline warning: offline and online frequently disagree, so nothing ships on offline metrics alone.

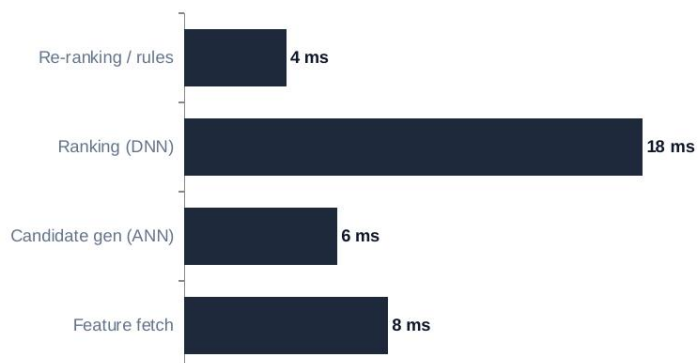
ABBREVIATIONS

- NDCG = Normalized Discounted Cumulative Gain
- MAP = Mean Average Precision
- AUC = Area Under the ROC Curve
- CTR = Click-Through Rate
- CVR = Conversion Rate
- AOV = Average Order Value

SERVING & LATENCY

Where the milliseconds go (and how to save them)

Illustrative per-request budget within a page-load SLA. Heavy compute is precomputed in batch.



Batch vs. real-time



Batch

Embeddings, ANN index, candidate lists, features



Real-time

Feature fetch, ranking, re-ranking, fallback



Levers

Caching, model distillation, quantization, early-exit

Speaker notes

Make latency concrete with a budget breakdown. Ranking usually dominates, which is why you push everything possible into batch (embeddings, ANN index, candidate lists) and keep the online path lean. Optimization levers when you blow the budget: cache hot results, distill or quantize the ranker, and use early-exit/cascade ranking. The numbers here are illustrative — replace with your traced latencies.

MODIFIABLE FOR YOUR PLATFORM

- Replace the per-stage millisecond values with real numbers from your tracing/profiling.
- State your actual latency SLA (e.g., p99).

ABBREVIATIONS

- SLA = Service-Level Agreement (latency target)
- ANN = Approximate Nearest Neighbor
- DNN = Deep Neural Network
- Quantization = reducing numeric precision to speed up inference

PRODUCTION CASE STUDIES

How leaders solved scale, sparsity & cold start



Taobao / Alibaba

EGES: build an item graph from behavior, learn embeddings, fold in side info (category, brand, price) for cold start.

A/B tested CTR gains on the Mobile Taobao home feed at billion-item scale.



JD.com (Jingdong)

Deep semantic retrieval + a pairwise deep re-ranker for product search and recommendations.

Millisecond retrieval and personalized ranking across a \$100B+ GMV catalog.



Amazon

Item-to-item collaborative filtering: precompute an item-item similarity matrix offline.

O(1)-style online lookups; scales where user-based kNN does not.

Speaker notes

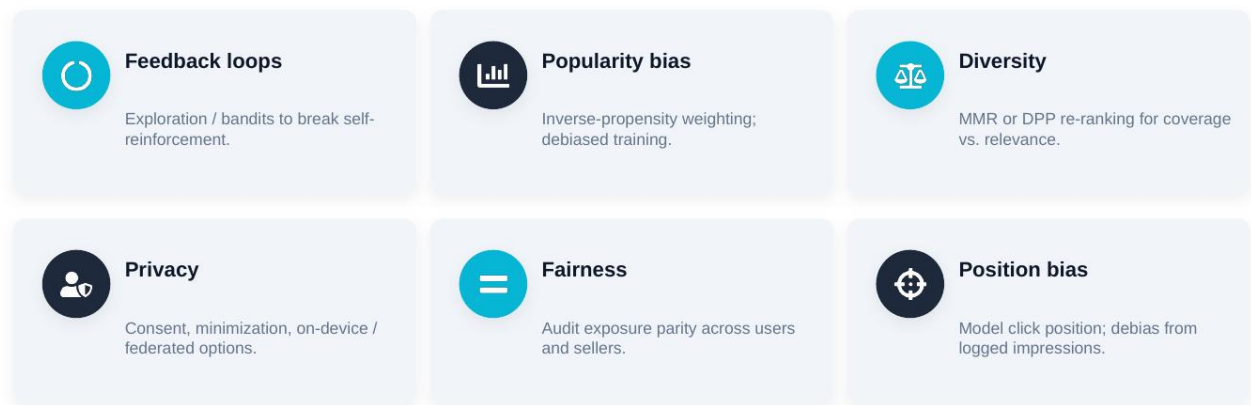
Ground the architecture in published practice. Alibaba's EGES (KDD 2018) is the reference for graph-embedding retrieval with side-information aggregation to fix sparsity and cold start; it was A/B-validated for CTR gains on Taobao's mobile home feed at billion-item scale. JD.com published deep semantic retrieval plus pairwise deep re-ranking for search, emphasizing millisecond latency at massive GMV. Amazon's item-to-item is the enduring lesson in decoupling offline similarity computation from cheap online lookup. All three reinforce our architecture choices.

ABBREVIATIONS

- EGES = Enhanced Graph Embedding with Side information
- CTR = Click-Through Rate
- GMV = Gross Merchandise Value
- kNN = k-Nearest Neighbors
- JD = Jingdong (JD.com)

CHALLENGES & MITIGATIONS

The hard problems, with concrete levers



Speaker notes

For engineers, name the lever, not just the problem. Feedback loops → exploration/bandits. Popularity and position bias → inverse-propensity weighting and debiased training from logged impressions (which is why impression logging matters). Diversity → MMR or DPP at re-rank. Privacy → consent, data minimization, and on-device/federated approaches. Fairness → exposure-parity audits across users and sellers. Each is an engineering workstream, not a slogan.

ABBREVIATIONS

- MMR = Maximal Marginal Relevance
- DPP = Determinantal Point Process (diversity-promoting selection)
- IPW = Inverse-Propensity Weighting

STACK & BUILD DECISIONS

A reference toolkit and the choices that matter

Layer	Options	Decision driver
Retrieval / ANN	FAISS · ScaNN · HNSWlib · vector DB	recall vs. latency vs. ops cost
Modeling	GBDT (XGBoost/LightGBM) · TensorFlow · PyTorch	team skills · model complexity
Features	Feature store (e.g. Feast) · streaming	train/serve parity · freshness
Serving	Model server · cache · autoscaling	p99 latency · cost
Experimentation	A/B platform · interleaving · metrics	trustworthy decisions

Build vs. buy: buy retrieval/feature infra to start; build the ranker where domain data is your edge. Roll out behind flags with A/B gating.

Speaker notes

Close the technical body with a decision table, not a tool worship list. For each layer the point is the decision driver, not the brand: retrieval is a recall/latency/ops tradeoff; modeling should match team skill and required complexity (don't reach for transformers if a GBDT hits the metric); features live or die on train/serve parity; serving is a p99/cost problem; experimentation must be trustworthy. Build-vs-buy: buy commodity infra, build where your proprietary data is the moat, and always roll out behind flags with A/B gating.

MODIFIABLE FOR YOUR PLATFORM

- Replace the Options column with your actual or proposed stack.
- Add a 'current state' column if this is a migration plan.

ABBREVIATIONS

- FAISS / ScaNN / HNSWlib = ANN libraries
- GBDT = Gradient-Boosted Decision Trees
- p99 = 99th-percentile latency

KEY TAKEAWAYS

Engineering takeaways

- 1 Recommendation = constrained top-K over $f(\text{user}, \text{item}, \text{context})$
- 2 Retrieval → ranking → re-ranking, with batch/real-time decoupling
- 3 Embeddings + ANN for recall; GBDT/DNN/sequence models for ranking
- 4 Treat implicit feedback as biased signal; fix train/serve skew
- 5 Offline metrics gate; online A/B decides — design for the gap

Let's align on the reference architecture and a build plan.

Thank you

Speaker notes

Summarize the system view and pivot to action: agree on the reference architecture, the initial retrieval+ranking choices, the evaluation harness, and a phased build plan. Offer to deep-dive any single stage (retrieval, ranking, serving, or eval) in a follow-up working session.

ABBREVIATIONS

- ANN = Approximate Nearest Neighbor
- GBDT = Gradient-Boosted Decision Trees
- DNN = Deep Neural Network

APPENDIX

Production systems deep-dive

R&D history (pre- vs post-GPT) and per-segment effectiveness for Amazon, Taobao / Alibaba, and JD.com. All quantitative figures from 2024–2026 publications.

Speaker notes

Appendix divider. These slides support the deck with each platform's documented recommender history — including the post-GPT (generative/LLM) era — and effectiveness by user class. Every quantitative figure is from a source published after 2024.

APPENDIX · PRODUCTION SYSTEMS

Amazon — recommendation system history



From item-to-item CF to a RAG-based generative assistant + commonsense knowledge graph.



Sources: Amazon Science (2024); Amazon Q4 2025 earnings; COSMO — ACM SIGMOD 2024; Linden et al., IEEE Internet Computing (2003, historical).

Speaker notes

Architecture arc: offline item-item similarity (1998/2003) → deep per-widget personalization (2017) → a custom shopping LLM on Bedrock with RAG over catalog/reviews and the COSMO commonsense knowledge graph (2024), then account-level memory conditioning (2025). Business signal: ~\$12B incremental (Q4'25), +60% purchase completion for the engaged cohort.

ABBREVIATIONS

- CF = Collaborative Filtering
- RAG = Retrieval-Augmented Generation
- GenAI = Generative AI

SOURCES

- Amazon Science (2024); Amazon Q4 2025 earnings; COSMO, ACM SIGMOD 2024; Linden et al., IEEE (2003).

APPENDIX · PRODUCTION SYSTEMS

Taobao / Alibaba — recommendation system history



From graph embeddings/DIN to LLM-based generative retrieval and ranking.



Sources: Wang et al., KDD 2018 (EGES); Zhou et al., KDD 2018 (DIN); Yan et al., 2025 (LUM); Jiang et al., 2025 (URM, arXiv:2502.03041); Yi et al., 2025 (RecGPT).

Speaker notes

EGES (graph embeddings + side info) and DIN (attention CTR) defined the pre-GPT stack. 2025 shifts: LUM establishes recommendation scaling laws over long user sequences (+2.9% CTR/+1.2% RPM); URM casts retrieval as LLM generation with multi-query representation and semantic IDs, tens-of-ms latency, lifting new-category share 18.8%→46.2%; RecGPT replaces log-fitting with intent reasoning across mining/tagging/retrieval/explanation.

ABBREVIATIONS

- EGES = Enhanced Graph Embedding w/ Side info
- DIN = Deep Interest Network
- CTR = Click-Through Rate
- RPM = Revenue Per Mille
- LUM = Large User Model
- URM = Universal Retrieval Model

SOURCES

- Yan et al., LUM (2025); Jiang et al., URM (2025, arXiv:2502.03041); Yi et al., RecGPT (2025); Wang et al., EGES (KDD 2018).

APPENDIX · PRODUCTION SYSTEMS

JD.com (Jingdong) — recommendation system history



From deep retrieval to generative CTR and generative retrieval in production.



Sources: Zhao et al. (Telepath), 2017; Li et al., 2021; Kong et al., 2025 (GenCTR, arXiv:2507.11246); JD GRAM & CFRAG, 2025.

Speaker notes

Telepath (vision-aware DL, 2017) and deep semantic retrieval + pairwise re-ranking (2021) → 2025 generative stack: GenCTR uses generative pretraining then transfers to a discriminative CTR head (mitigating one-epoch overfitting); GRAM is generative retrieval + alignment; CFRAG mixes recall logits with LLM embeddings. Fewer public per-segment metrics than peers.

ABBREVIATIONS

- CTR = Click-Through Rate
- RAG = Retrieval-Augmented Generation
- GRAM = Generative Retrieval and Alignment Model

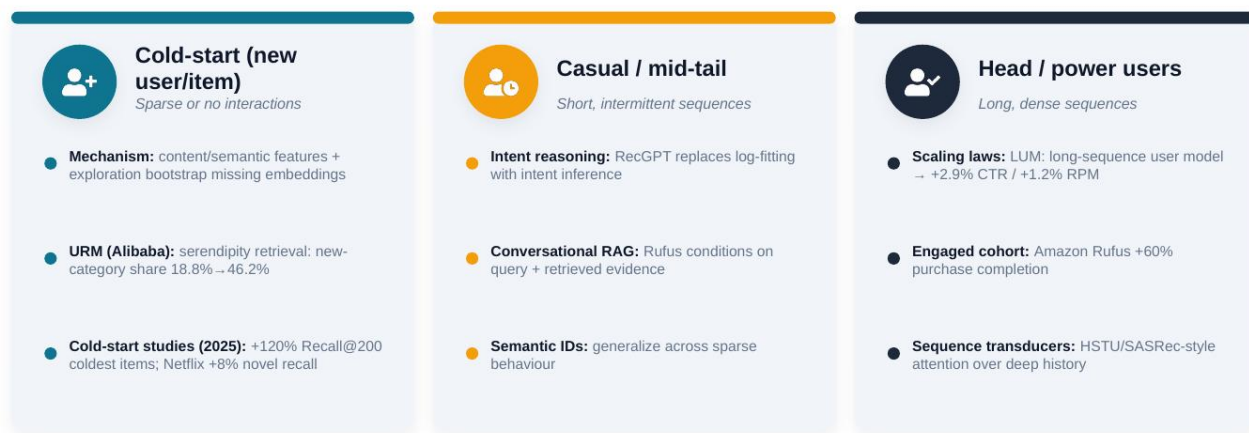
SOURCES

- Kong et al., GenCTR (2025, arXiv:2507.11246); JD GRAM & CFRAG (2025); Zhao et al. (2017); Li et al. (2021).

APPENDIX · PRODUCTION SYSTEMS

Effectiveness by user segment (LLM era)

Empirically, generative/LLM methods yield the largest lift on cold-start segments; power users benefit from long-sequence user modeling.



Sources (2025): Jiang et al. (URM); Yan et al. (LUM); Amazon Q4 2025; Li et al., Nov 2025 (Netflix); cold-start video study, Aug 2025. Platform-level per-segment figures are rarely disclosed; cross-industry studies used as directional evidence.

Speaker notes

Segment-wise: cold-start is where generative/LLM methods dominate classical CF, because semantic features and exploration bootstrap missing embeddings (URM serendipity 18.8%→46.2%; cold-start studies report large Recall@k gains). Mid-tail benefits from intent reasoning and RAG conditioning; head users benefit from long-sequence user models and scaling laws (LUM). Caveat: platform per-segment disclosures are rare; figures are drawn from 2025 papers as directional evidence.

ABBREVIATIONS

- CF = Collaborative Filtering
- Recall@k = share of relevant items in top k
- CTR = Click-Through Rate
- RAG = Retrieval-Augmented Generation
- URM = Universal Retrieval Model
- LUM = Large User Model

SOURCES

- Jiang et al., URM (2025); Yan et al., LUM (2025); Amazon Q4 2025; Li et al., Nov 2025 (Netflix cold-start); cold-start video study, Aug 2025.

APPENDIX · SOURCES

LLM-era sources (2024–2026)

Amazon — Rufus	Amazon Science, “The technology behind Amazon’s GenAI shopping assistant, Rufus” (2024); scale & sales via Amazon Q4 2025 earnings. <i>amazon.science/blog/...-rufus</i>
Amazon — COSMO	Commonsense knowledge graph built with LLMs, ACM SIGMOD 2024. <i>amazon.science (COSMO, SIGMOD 2024)</i>
Alibaba — URM	Jiang et al., “LLM as Universal Retriever in Industrial-Scale Recommender System” (2025). <i>arxiv.org/abs/2502.03041</i>
Alibaba — LUM & RecGPT	Yan et al., “Large User Model” (2025); Yi et al., “RecGPT Technical Report” (2025). <i>arXiv 2025 (LUM); RecGPT report, Aug 2025</i>
JD.com — GenCTR & GRAM	Kong et al., “Generative CTR Prediction...” (2025); JD “GRAM: Generative Retrieval and Alignment” (2025). <i>arxiv.org/abs/2507.11246</i>
User-class / cold-start	Netflix cold-start (Li et al., Nov 2025); video-platform cold-start study (Aug 2025); cold-start LLM survey (2025). <i>arXiv 2025</i>

Note: pre-GPT milestones (item-to-item CF 1998/2003; EGES & DIN 2018; JD Telepath 2017) are historical; all quantitative effectiveness figures above are from 2024–2026 publications.

Speaker notes

Consolidated citations for the appendix. All quantitative effectiveness numbers come from 2024–2026 sources; the only pre-2024 references describe historical milestones.